Third-Party Application Patching with Intune

Implementation Solution Guide

Version 1.0.0 · 2026-06-28

Contents

Executive Summary — Third-Party Application Patching with Intune (Microsoft Intune) 4
- What you get . . . 4
- What this delivers . . . 4
- The repeatable deployment model . . . 4
- Outcome . . . 4

1. Solution Overview — Third-Party Application Patching with Intune 5
- What this solution is . . . 5
- Why third-party apps are the prime attack surface . . . 5
- The two patching paths . . . 5
- How it works end to end . . . 5
- What you can control . . . 6
- Benefits . . . 6
- Limitations (be honest with buyers) . . . 6
- The standard deployment model . . . 7

2. Requirements Guide — Third-Party Application Patching with Intune 7
- 2.1 Licensing . . . 7
- 2.2 Device enrollment . . . 7
- 2.3 App inventory (what is installed) . . . 7
- 2.4 Enterprise App Catalog availability . . . 7
- 2.5 Packaging tooling for non-catalog apps . . . 7
- 2.6 Detection method per packaged app . . . 8
- 2.7 System-context install (no per-user blockers) . . . 8
- 2.8 Maintenance windows / restart expectations . . . 8
- Checklist . . . 8

Global Requirements — Intune Solutions (category-level) 8
- G1. Microsoft Entra ID tenant . . . 8
- G2. Microsoft Intune subscription . . . 9
- G3. Administrative roles . . . 9
- G4. Device enrollment in Intune . . . 9
- G5. Pilot security group . . . 9
- G6. Supported Windows baseline . . . 9
- G7. Network access to Microsoft cloud endpoints . . . 9

3. Implementation Guide — Third-Party Application Patching with Intune 10
- Step 1 — Create the ring groups . . . 10
- Step 2 — Inventory what is installed . . . 10

Step 3 — Path A: onboard Enterprise App Catalog apps (auto-update) . . . 10
Step 4 — Path B: package a non-catalog app as a superseding Win32 update . . . 11
Step 5 — Apply on a pilot device . . . 11
Step 6 — Validate the pilot . . . 12
Step 7 — Expand to production . . . 12
Step 8 — Each new release (repeat) . . . 12
5. Testing Guide — Third-Party Application Patching with Intune 12
Test 1 — Catalog app deployed . . . 12
Test 2 — Win32 update deployed . . . 12
Test 3 — Installed version meets target on device . . . 13
Test 4 — Detection rule is correct . . . 13
Test 5 — Supersedence moves devices forward . . . 13
Test 6 — Fleet exposure report . . . 13
Test 7 — Stuck-update remediation (safe) . . . 13
Pass/fail gate . . . 13
6. Troubleshooting Guide — Third-Party Application Patching with Intune 13
A. Update not applying / app stays on old version . . . 14
B. Update re-installs every cycle (never "done") . . . 14
C. Catalog app issues . . . 14
D. Install fails because the app is open . . . 14
E. Conflicting updater . . . 14
F. False exposure in the report . . . 14
G. Diagnostics quick reference . . . 15
H. When to escalate . . . 15
7. Best Practices — Third-Party Application Patching with Intune 15
8. Deliverables Summary — Third-Party Application Patching with Intune 16
Documentation . . . 16
PowerShell scripts . . . 16
What the buyer receives . . . 16
Outcome . . . 17
Appendix A — PowerShell Scripts 18

Executive Summary — Third-Party Application Patching with Intune (Microsoft Intune)

Windows Update keeps the operating system current, but the software that attackers exploit most — browsers, PDF readers, Java, conferencing clients, archive utilities — quietly drifts out of date on every device. Each stale version is an open door, and most organizations have no fleet-wide view of which apps are behind. This package gives you a **repeatable third-party patching program in Microsoft Intune**: keep non-Microsoft apps current automatically, using the Enterprise App Catalog where the app is available, and a packaged update method (detection plus supersedence) for everything else, deployed in rings.

This is a complete, **deploy-it-today** package, written for an administrator with no prior app-packaging experience. Every step is click-by-click against the current Intune admin center.

What you get

- A plain-language **Solution Overview** of why third-party apps are the prime attack surface, the

Enterprise App Catalog (catalog apps with auto-update) model, and the packaged/superseded Win32 update method for everything else.

- A **Requirements Guide** with verify / implement-if-missing for every prerequisite.
- An **Implementation Guide** of precise, ordered Intune steps covering both update paths.
- Five **PowerShell scripts** for installed-app discovery, patch-state validation, fleet exposure

reporting, safe remediation of stuck updates, and an update-ring group builder.

- **Testing** and **Troubleshooting** guides for confident rollout.

What this delivers

A **ring-based third-party patching strategy** — catalog apps onboarded with auto-update enabled, non-catalog apps packaged as Win32 updates with detection rules and supersedence so devices move to the current version automatically, all assigned through a fast **Pilot** ring and a slower **Broad** ring, with a fleet exposure report that shows what is still out of date.

The repeatable deployment model

Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

Your highest-risk third-party apps kept current automatically across the fleet — catalog apps auto-updating, everything else packaged and superseded, deployed in rings — proven on a pilot before you scale.

1. Solution Overview — Third-Party Application Patching with Intune

What this solution is

Microsoft Intune can keep **non-Microsoft applications** current the same way it manages everything else: as cloud-delivered apps assigned to devices. This solution builds a repeatable program that patches the third-party software attackers target most — browsers, PDF readers, Java, conferencing clients, archive tools — using two complementary paths inside Intune, deployed in update rings so a small pilot validates every change before the fleet receives it.

Why third-party apps are the prime attack surface

Windows Update for Business patches the OS, Microsoft drivers, and Microsoft Store apps — but **not** the third-party software installed on top. Those apps publish their own security fixes on their own cadence, and without a managed program they drift: a device can be fully patched on Windows yet run a browser or PDF reader months behind, with public CVEs against it. Because these apps render untrusted content from the internet, they are the most commonly exploited software on a managed device. A patching program for them closes doors that OS patching alone leaves open.

The two patching paths

Intune offers two ways to keep a third-party app current. Use the catalog path wherever the app is available, and the packaged path for everything else.

Path A — Enterprise App Catalog apps (with auto-update)

The **Enterprise App Catalog** is a Microsoft-curated, prepackaged set of common third-party Win32 apps. When you add an app from the catalog, Intune supplies the installer, detection, and update logic; you can turn on **auto-update** so Intune ships new versions of that app to assigned devices as the catalog publishes them — no repackaging on your part. This is the lowest-effort path and is preferred whenever the app you need is in the catalog.

Path B — Packaged Win32 updates (detection + supersedence)

For apps not in the catalog, you keep them current with the **Win32 app** model. You package the new version as a Win32 app (`.intunewin`), give it a **detection rule** that returns "installed" only when the current (or newer) version is present, and mark it to **supersede** the prior Win32 app so assigned devices automatically replace the old version with the new one. Each time the vendor ships a release, you package it and supersede the previous package; devices move forward on their next cycle.

How it works end to end

1. **Inventory** the third-party apps and versions actually installed across the fleet (the

discovery script reads installed apps; Intune's discovered-apps inventory corroborates).

1. **Onboard catalog apps** from the Enterprise App Catalog with auto-update enabled, assigned by

ring.

1. **Package non-catalog apps** as Win32 updates with a version-aware detection rule and supersedence

of the prior package, assigned by ring.

1. **Assign in rings** — a fast **Pilot** ring gets each change first; a slower **Broad** ring

follows once the pilot is healthy.

1. **Validate** patch state on devices and run a fleet **exposure report** to find what is still out

of date.

1. **Expand** to the broad ring, then repeat each release cycle.

What you can control

- Which apps are managed, and via which path (catalog auto-update vs packaged supersedence).
- The exact target version a device must reach (through the detection rule / catalog version).
- The rollout order and pace (Pilot then Broad rings, with a gate between them).
- Install behavior (system context, restart handling) for packaged updates.
- Fleet-wide visibility of out-of-date exposure.

Benefits

- **Security** — the most-exploited software on each device stays current on a predictable cadence.
- **Low effort where it counts** — catalog apps auto-update with no repackaging.
- **Automatic version moves** — supersedence pulls devices off old versions without manual cleanup.
- **Safe rollout** — ring-based deployment catches a bad update on the pilot before the fleet.
- **Visibility** — a fleet exposure report turns "we think we're patched" into evidence.

Limitations (be honest with buyers)

- The **Enterprise App Catalog** covers many common apps but not every app; non-catalog apps require

the packaged path and a tool to create the `.intunewin` package.

- Intune reports installed apps via **discovered apps**, but it does not natively scan each app

against the latest vendor version; the exposure report compares installed versions against a target you supply.

- Packaged Win32 updates require a correct, version-aware **detection rule** — a weak detection rule

is the most common cause of an update that never applies or re-applies forever.

- Some apps include their own auto-updater; decide per app whether Intune or the app's updater owns

patching to avoid two authorities fighting.

The standard deployment model

> ***Inventory installed apps -> onboard catalog apps (auto-update) and package non-catalog updates (detection + supersedence) -> assign by ring -> validate on the pilot -> expand to broad.*** *The repeatable way:* ***Assess -> Configure -> Pilot -> Validate -> Expand.***

2. Requirements Guide — Third-Party Application Patching with Intune

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

2.1 Licensing

Description: Microsoft Intune (Plan 1), included in M365 E3/E5 or Business Premium. **Why:** Intune delivers the catalog apps and Win32 app deployments that patch third-party software. **Verify:** M365 admin center -> Billing -> Licenses; Intune is present and assigned. **Implement:** Assign M365 E3/E5 or Business Premium (Intune included), or standalone Intune Plan 1.

2.2 Device enrollment

Description: Targeted Windows devices enrolled in Intune. **Why:** Catalog apps and Win32 updates apply only to enrolled devices. **Verify:** Intune -> Devices -> All devices; the target devices appear and are managed. **Implement:** Enable automatic enrollment and join devices to Entra ID.

2.3 App inventory (what is installed)

Description: A current inventory of the third-party apps and versions installed across the fleet. **Why:** You cannot patch what you have not inventoried; the inventory drives which apps to manage. **Verify:** Intune -> Apps -> Monitor -> Discovered apps; corroborate on a device with `TPP-Discovery.ps1`. **Implement:** Run `TPP-Discovery.ps1` on representative devices and review Discovered apps in Intune.

2.4 Enterprise App Catalog availability

Description: Access to the Enterprise App Catalog and confirmation of which of your apps it covers. **Why:** The catalog path (auto-update, no repackaging) is preferred wherever the app is available. **Verify:** Intune -> Apps -> Windows -> Add -> app type **Enterprise App Catalog app**; search for each target app. **Implement:** For apps in the catalog, use Path A; for apps not in the catalog, plan the packaged path (2.5).

2.5 Packaging tooling for non-catalog apps

Description: The Microsoft Win32 Content Prep Tool to create `.intunewin` packages, plus each app's installer. **Why:** Apps not in the catalog are patched as Win32 update apps, which require a packaged `.intunewin`. **Verify:** `IntuneWinAppUtil.exe` is available; you can obtain each app's silent installer and version. **Implement:** Download the Win32 Content Prep Tool from Microsoft; collect vendor silent

installers.

2.6 Detection method per packaged app

Description: A reliable, version-aware way to detect whether the target version is installed. **Why:** Supersedence and "installed" status depend on a detection rule that recognizes the current version. **Verify:** For each app, identify a registry uninstall key (DisplayVersion), a file version, or an MSI product code. **Implement:** Record the detection signal per app (see Implementation Step 4) before packaging.

2.7 System-context install (no per-user blockers)

Description: Packaged updates install in system context; apps must close or update silently. **Why:** An app held open by the user, or an installer that needs the app closed, can block the update. **Verify:** Test the silent install/update command on a device with the app installed. **Implement:** Use the vendor's silent update switches; for apps that must be closed, plan a restart/maintenance behavior.

2.8 Maintenance windows / restart expectations

Description: Agreement on when updates may install and restart apps for users. **Why:** Replacing a running app may require closing it; setting expectations reduces tickets. **Verify:** Confirm with the business which rings tolerate mid-day app restarts. **Implement:** Communicate the cadence; use the Pilot ring to validate user impact before Broad.

Checklist

- Intune licensing assigned
- Devices enrolled
- App inventory captured — `TPP-Discovery.ps1` + Discovered apps
- Enterprise App Catalog coverage confirmed per app
- Win32 Content Prep Tool + vendor installers for non-catalog apps
- Detection signal recorded per packaged app
- Silent install/update validated in system context
- Maintenance/restart expectations agreed
- Pilot + Broad Entra groups created

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** Phoenixtekk solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every Phoenixtekk solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The Phoenixtekk deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Third-Party Application Patching with Intune

Follow these steps **in order**, in the **Microsoft Intune admin center** (`https://intune.microsoft.com`).

> ***Model:*** *Inventory installed apps, onboard catalog apps with auto-update, package non-catalog apps as superseding Win32 updates, then assign everything through a fast* ***Pilot*** *ring and a slower* ***Broad*** *ring. The pilot validates every change before the fleet receives it.* ***Naming convention (reuse verbatim):*** *every object is named* `<Purpose> - <Scope> - <Stage>`*. Example apps below use a browser and a PDF reader with placeholder versions in backticks; substitute your real apps and the actual current version when you build them.*

Step 1 — Create the ring groups

1. Entra -> Groups -> New group -> Security -> name `Third-Party Patching - Devices - Pilot` (a few representative devices). Create.
2. Create `Third-Party Patching - Devices - Broad` (the rest of the fleet). Create.
3. Tip: `TPP-Set-UpdateRingTargeting.ps1` can build these groups and seed pilot membership for you.

Step 2 — Inventory what is installed

1. Run `TPP-Discovery.ps1` on representative devices to list installed third-party apps and versions.
2. In Intune -> Apps -> Monitor -> Discovered apps, corroborate the fleet-wide picture.
3. For each app you will manage, decide the path: **Path A** if it is in the Enterprise App Catalog (Step 3), otherwise **Path B** packaged update (Step 4).

Step 3 — Path A: onboard Enterprise App Catalog apps (auto-update)

Do this for each app available in the catalog (example: a browser).

1. Intune -> Apps -> Windows -> Add. For **App type**, choose **Enterprise App Catalog app**. Select.
2. **Search the catalog** for the app, select the package, then Select.
3. On **App information**, keep the catalog name; set a clear display name if you wish. Next.
4. On **Program / settings**, leave the catalog-provided install/uninstall commands. **Enable auto-update** (Automatically update this app to the latest version) so Intune ships new versions as the catalog publishes them. Next through Detection (catalog-provided) and Requirements.
1. **Assignments -> Required -> Add group ->** `Third-Party Patching - Devices - Pilot`. Next.
2. Review + create. Name the assignment context so it is recognizable: this is your `Browser Catalog Update - Catalog - Pilot` deployment.

1. After the pilot validates (Testing Guide), add the **Required** assignment to

`Third-Party Patching - Devices - Broad` — call this `Browser Catalog Update - Catalog - Broad`.

Step 4 — Path B: package a non-catalog app as a superseding Win32 update

Do this for each app **not** in the catalog (example: a PDF reader at target version <x.y.z>).

1. **Package it.** With the Win32 Content Prep Tool, wrap the vendor silent installer for version

<x.y.z> into an `.intunewin` file.

1. Intune -> Apps -> Windows -> Add -> **App type: Windows app (Win32)**. Select. Upload the `.intunewin`.

2. **App information:** display name `PDF Reader Update - Win32 -` <x.y.z> (include the version so

each release is distinct). Next.

1. **Program:** enter the vendor **silent install** command and the **uninstall** command. Set

Install behavior: System. Next.

1. **Requirements:** Operating system architecture and minimum OS. Next.

2. **Detection rules:** add a **version-aware** rule so the app is "installed" only when the target

version is present. Use one of:

• Registry: the app's uninstall key `DisplayVersion` value, operator **Version greater than or equal to** <x.y.z>; or

• File: the installed binary's version, **greater than or equal to** <x.y.z>; or

• MSI: the product code with the matching version.

Name this rule mentally as `PDF Reader Detection - Version -` <x.y.z> for your runbook. Next.

1. **Supersedence:** add the **previous** PDF reader Win32 package as superseded, with **Update**

(replace the old version). This makes assigned devices move from the old version to <x.y.z> automatically. (On the very first package there is nothing to supersede yet.) Next.

1. **Assignments -> Required -> Add group ->** `Third-Party Patching - Devices - Pilot`. Review + create.

This is your `PDF Reader Update - Win32 - Pilot` deployment.

1. After the pilot validates, add the **Required** assignment to `Third-Party Patching - Devices - Broad`

(`PDF Reader Update - Win32 - Broad`).

Step 5 — Apply on a pilot device

1. On a pilot device, sync policy (Settings -> Accounts -> Access work or school -> Info -> Sync).

2. Let the catalog app update and the Win32 update install.

3. Run `TPP-Validation.ps1` to confirm the installed versions now meet the targets.

Step 6 — Validate the pilot

1. Run `TPP-Validation.ps1` on pilot devices -> confirms each managed app is at (or above) its target version.

2. Run `TPP-Report.ps1` -> confirms pilot devices report no out-of-date exposure for managed apps.

3. In Intune -> Apps -> Monitor, confirm the catalog app and Win32 update show **Installed/Succeeded**

on the pilot. Complete the Testing Guide.

Step 7 — Expand to production

1. Add the **Broad** assignments (Steps 3.7 and 4.9) once the pilot passes.

2. Re-run `TPP-Report.ps1` to confirm broad devices converge to the target versions.

3. Use `TPP-Remediation.ps1` (default `-WhatIf`) to nudge any device whose update is stuck.

Step 8 — Each new release (repeat)

1. For **catalog apps**, auto-update handles new versions automatically — nothing to do.

2. For **packaged apps**, package the new version as `App Update - Win32 - <newversion>`, point its

detection rule at the new version, **supersede** the prior package, and assign Pilot then Broad.

1. Re-validate each cycle with `TPP-Validation.ps1` and `TPP-Report.ps1`.

Order of operations

```
Ring groups -> Inventory -> Catalog apps (auto-update) + Win32 packages
(detection + supersedence) -> Assign Pilot -> Validate -> Expand to Broad ->
Repeat per release
```

5. Testing Guide — Third-Party Application Patching with Intune

Run after the pilot assignments apply (Implementation Step 5-6) and after each release cycle.

Test 1 — Catalog app deployed

Do: Intune -> Apps -> Windows -> open the Enterprise App Catalog app -> Device install status.
Expected: Pilot devices show **Installed**; auto-update is enabled on the app.

Test 2 — Win32 update deployed

Do: Intune -> Apps -> Windows -> open `PDF Reader Update - Win32 - <x.y.z>` -> Device install status. **Expected:** Pilot devices show **Installed / Succeeded**; the superseded prior package shows replaced.

Test 3 — Installed version meets target on device

Do: On a pilot device run `TPP-Validation.ps1`. **Expected:** Each managed app reports a version greater than or equal to its target — result **PATCH STATE OK**.

Test 4 — Detection rule is correct

Do: Confirm the Win32 app reports Installed only when the target version is present (check a device already at the version, and one below it). **Expected:** Devices at/above `<x.y.z>` = Installed; devices below = the update applies, then Installed.

Test 5 — Supersedence moves devices forward

Do: On a device running the old version, sync and let the update run. **Expected:** The old version is replaced by the target version; `TPP-Validation.ps1` confirms the new version.

Test 6 — Fleet exposure report

Do: Run `TPP-Report.ps1` against the fleet with your target versions. **Expected:** A clear per-app / per-device list of devices below target (out-of-date exposure) and the count current.

Test 7 — Stuck-update remediation (safe)

Do: On a device whose update did not apply, run `TPP-Remediation.ps1` (default `-WhatIf` to preview), then with `-WhatIf:$false` to act. **Expected:** `-WhatIf` lists the actions it would take; the live run requests a sync/app re-evaluation, after which the update applies.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-5 pass; managed apps reach their target version on the pilot
Broad pass	Tests 1-3, 6 pass for the broad ring before the next release is promoted

6. Troubleshooting Guide — Third-Party Application Patching with Intune

Use `TPP-Discovery.ps1` / `TPP-Validation.ps1` for device state, `TPP-Report.ps1` for fleet exposure, and `TPP-Remediation.ps1` (default `-WhatIf`) for safe fixes.

A. Update not applying / app stays on old version

Symptom	Cause	Remediation
Device never updates the app	Not synced, or assignment not received	Run `TPP-Remediation.ps1` (sync + app re-evaluation); confirm group membership
Win32 update shows "Installed" but version is old	Detection rule matches too loosely (not version-aware)	Re-key detection to `DisplayVersion` / file version greater-than-or-equal-to the target
Old version not removed	Supersedence not set, or set to install-alongside	Edit the new package supersedence to **Update/replace** the prior package

B. Update re-installs every cycle (never "done")

- The detection rule does not recognize the version it just installed. Confirm the rule reads the same `DisplayVersion` / file path the installer actually writes, with the correct operator and value.

C. Catalog app issues

Symptom	Cause	Remediation
Catalog app not updating to new version	Auto-update not enabled on the app	Edit the Enterprise App Catalog app and enable automatic update
App not found in catalog	App not covered by the Enterprise App Catalog	Use the packaged Win32 path (Implementation Step 4)

D. Install fails because the app is open

- Installers that require the app closed will fail in system context while the user has it running.

Use the vendor's in-use / silent-update switches, or schedule the update for a maintenance window and communicate the restart to users.

E. Conflicting updater

- An app with its own auto-updater can fight Intune (versions bounce, false "out of date"). Decide one authority per app: disable the app's updater where Intune owns patching, or exclude that app from Intune patching where its own updater owns it.

F. False exposure in the report

- `TPP-Report.ps1` compares installed versions to the target you supply. A wrong target string, or an app whose `DisplayVersion` format differs, can show false exposure. Confirm the target and the observed `DisplayVersion` with `TPP-Discovery.ps1`.

G. Diagnostics quick reference

```
# installed third-party apps + versions (both registry hives)
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*','HKLM:\SOFTWARE\WOW6432Node
  Where-Object DisplayName | Select-Object DisplayName, DisplayVersion, Publisher
# force a management sync so assignments re-apply
Start-Process "$env:WINDIR\System32\deviceenroller.exe" -ArgumentList "/c /AutoEnrollMDM" -WindowStyle Hid
```

Win32 app logs: `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log`.

H. When to escalate

If the assignment is received, supersedence is set, detection is version-aware, the device is synced, and the update still will not apply, collect the IntuneManagementExtension log and open a Microsoft support case.

7. Best Practices — Third-Party Application Patching with Intune

- **Prefer the catalog.** Wherever an app is in the Enterprise App Catalog, use it with **auto-update** — it is the lowest-effort path and removes per-release repackaging entirely.
- **Use rings, always.** A small fast **Pilot** ring catches a bad app update before it reaches the **Broad** fleet — the single most important practice.
- **Make detection version-aware.** A Win32 update is only as reliable as its detection rule; key it to `DisplayVersion` / file version greater-than-or-equal-to the target so it applies once and stops.
- **Supersede, do not stack.** Mark each new package to supersede and replace the prior one so devices move off old versions automatically instead of accumulating installs.
- **Name everything consistently.** Use `<Purpose> - <Scope> - <Stage>` for every group, app, and update (e.g. `PDF Reader Update - Win32 - Broad`) so the next release and the next admin are obvious.
- **Pick one updater per app.** If an app has its own auto-updater, decide whether Intune or the app owns patching — running both invites conflicts and false "out of date" reports.
- **Patch the high-risk apps first.** Browsers, PDF readers, Java, and conferencing clients render untrusted content and are exploited most; manage them before lower-risk utilities.
- **Report on exposure.** Run `TPP-Report.ps1` each cycle to turn "we think we're patched" into a per-device list of what is still behind, and chase the laggards.
- **Validate silent installs in system context.** Confirm the vendor silent switches work with the app closed before assigning broadly; an installer that needs the app closed will stall otherwise.

• **Document the cadence.** Record each managed app, its path, target version, and ring assignments for audits and repeatability.

8. Deliverables Summary — Third-Party Application Patching with Intune

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Why third-party apps are the attack surface; the catalog (auto-update) and packaged (detection + supersedence) paths; the ring model
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered Intune steps for both update paths, assigned by ring
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`TPP-Discovery.ps1`	Inventories installed third-party apps and versions from the registry uninstall keys
`TPP-Validation.ps1`	Confirms managed apps on a device meet their target versions (patch-state validation)
`TPP-Report.ps1`	Fleet-wide out-of-date exposure across managed devices (Microsoft Graph + targets)
`TPP-Remediation.ps1`	Safe remediation (default `-WhatIf`); nudges stuck app updates via sync / re-evaluation
`TPP-Set-UpdateRingTargeting.ps1`	Builds the Pilot/Broad update-ring groups and seeds membership (Microsoft Graph)

What the buyer receives

• This implementation guide as a **Word** document and **PDF**.

- All five **PowerShell scripts** in a single ZIP.
- A repeatable process: **Assess -> Configure -> Pilot -> Validate -> Expand.**

Outcome

The highest-risk third-party apps kept current automatically across the fleet — catalog apps auto-updating, everything else packaged and superseded, deployed in rings, with a fleet exposure report — proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

TPP-Discovery.ps1

Purpose: Reads the Windows uninstall registry keys (both 64-bit and 32-bit hives) to list every installed application with its DisplayVersion and Publisher, so you can see which third-party apps and versions are present and decide what to manage in Intune. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\TPP-Discovery.ps1 Optional: -CsvPath C:\Temp\tpp-installed.csv to export the inventory. Optional: -ExcludeMicrosoft to hide Microsoft-published apps. EXPECTED RESULTS A table of installed apps (Name, Version, Publisher) and an optional CSV. Read-only. .NOTES Workplace Marketplace — Intune Solutions. Read-only (no changes are made). #> [CmdletBinding()] param([string]$CsvPath, [switch]$ExcludeMicrosoft)

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\TPP-Discovery.ps1 Optional: -CsvPath C:\Temp\tpp-installed.csv to export the inventory. Optional: -ExcludeMicrosoft to hide Microsoft-published apps. EXPECTED RESULTS A table of installed apps (Name, Version, Publisher) and an optional CSV. Read-only. .NOTES Workplace Marketplace — Intune Solutions. Read-only (no changes are made). #> [CmdletBinding()] param([string]$CsvPath, [switch]$ExcludeMicrosoft)

TPP-Remediation.ps1

Purpose: Forces an Intune/MDM sync so app assignments re-apply and restarts the Intune Management Extension service so it re-evaluates Win32 app installs (the mechanism that delivers packaged third-party updates). Runs in preview mode by default and changes nothing until you opt in. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\TPP-Remediation.ps1 Act: powershell -ExecutionPolicy Bypass -File .\TPP-Remediation.ps1 -WhatIf:$false Run elevated. After it acts, allow a policy cycle, then re-run TPP-Validation.ps1. EXPECTED RESULTS With -WhatIf (default): a list of the actions it WOULD take, no changes made. With -WhatIf:$false: an MDM sync requested and the Intune Management Extension restarted. .NOTES Workplace Marketplace — Intune Solutions. Defaults to -WhatIf (no changes). #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param()

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\TPP-Remediation.ps1 Act: powershell -ExecutionPolicy Bypass -File .\TPP-Remediation.ps1 -WhatIf:$false Run elevated. After it acts, allow a policy cycle, then re-run TPP-Validation.ps1. EXPECTED RESULTS With -WhatIf (default): a list of the actions it WOULD take, no changes made. With -WhatIf:$false: an MDM sync requested and the Intune Management Extension restarted. .NOTES Workplace Marketplace — Intune Solutions. Defaults to -WhatIf (no changes). #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param()

TPP-Report.ps1

Purpose: Pulls Intune's discovered-apps inventory across managed Windows devices and, for each app you manage, compares the detected version against a target you supply — producing a per-app / per-device list of devices that are below target (out-of-date exposure) and a current count. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TPP-Report.ps1 -CsvPath C:\Temp\tpp-exposure.csv Requires the Microsoft Graph PowerShell SDK and the scopes: DeviceManagementManagedDevices.Read.All Edit

the $Targets table below (or pass -TargetsCsv with columns NameLike,MinVersion). EXPECTED RESULTS A summary of devices below target per managed app, plus an optional per-device CSV. Read-only. .NOTES Workplace Marketplace — Intune Solutions. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath, [string]$TargetsCsv)

How to run: powershell -ExecutionPolicy Bypass -File .\TPP-Report.ps1 -CsvPath C:\Temp\tpp-exposure.csv Requires the Microsoft Graph PowerShell SDK and the scopes: DeviceManagementManagedDevices.Read.All Edit the $Targets table below (or pass -TargetsCsv with columns NameLike,MinVersion). EXPECTED RESULTS A summary of devices below target per managed app, plus an optional per-device CSV. Read-only. .NOTES Workplace Marketplace — Intune Solutions. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath, [string]$TargetsCsv)

TPP-Set-UpdateRingTargeting.ps1

Purpose: Creates the two Microsoft Entra security groups used to ring third-party app assignments and (optionally) seeds the Pilot group with a set of device names, so catalog auto-update apps and packaged Win32 updates can be assigned Pilot-first, then Broad. Uses the naming convention " - - ". HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TPP-Set-UpdateRingTargeting.ps1 Optional: -PilotDeviceNames 'PC-001','PC-002' to seed the Pilot group with those devices. Optional: -WhatIf to preview without creating anything. Requires the Microsoft Graph PowerShell SDK and the scopes: Group.ReadWrite.All, Device.Read.All EXPECTED RESULTS The two groups exist (created if missing): Third-Party Patching - Devices - Pilot Third-Party Patching - Devices - Broad and, if -PilotDeviceNames is supplied, those devices are added to the Pilot group. Assign your catalog apps and Win32 updates to these groups in Intune (Implementation Steps 3-4). .NOTES Workplace Marketplace — Intune Solutions. Supports -WhatIf. #> [CmdletBinding(SupportsShouldProcess = $true)] param([string[]]$PilotDeviceNames)

How to run: powershell -ExecutionPolicy Bypass -File .\TPP-Set-UpdateRingTargeting.ps1 Optional: -PilotDeviceNames 'PC-001','PC-002' to seed the Pilot group with those devices. Optional: -WhatIf to preview without creating anything. Requires the Microsoft Graph PowerShell SDK and the scopes: Group.ReadWrite.All, Device.Read.All EXPECTED RESULTS The two groups exist (created if missing): Third-Party Patching - Devices - Pilot Third-Party Patching - Devices - Broad and, if -PilotDeviceNames is supplied, those devices are added to the Pilot group. Assign your catalog apps and Win32 updates to these groups in Intune (Implementation Steps 3-4). .NOTES Workplace Marketplace — Intune Solutions. Supports -WhatIf. #> [CmdletBinding(SupportsShouldProcess = $true)] param([string[]]$PilotDeviceNames)

TPP-Validation.ps1

Purpose: Compares the installed version of each managed app (from the uninstall registry keys) against a target version you supply, and reports whether the device's third-party patch state is OK. This confirms a catalog auto-update or a packaged Win32 update actually reached the target. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\TPP-Validation.ps1 Edit the $Targets table below (or pass -TargetsCsv with columns NameLike,MinVersion) to list the apps you manage and their required minimum versions. Example CSV row: NameLike,MinVersion *Reader*,24.1.0 EXPECTED RESULTS Per-app PASS/FAIL and an overall result: "PATCH STATE OK" (exit 0) or "OUT OF DATE" (exit 1). .NOTES Workplace Marketplace — Intune Solutions. Read-only (no changes are made). #> [CmdletBinding()] param([string]$TargetsCsv)

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\TPP-Validation.ps1 Edit the $Targets table below (or pass -TargetsCsv with columns NameLike,MinVersion) to list the apps you manage and their required minimum versions. Example CSV row: NameLike,MinVersion *Reader*,24.1.0 EXPECTED RESULTS Per-app PASS/FAIL and an overall result: "PATCH STATE OK" (exit 0) or "OUT OF DATE" (exit 1). .NOTES Workplace Marketplace — Intune Solutions. Read-only (no changes are made). #> [CmdletBinding()] param([string]$TargetsCsv)